

Event, State, dan Implisit Intent pada Jetpack Compose

Tujuan Pembelajaran

1. Memahami konsep event handling pada Jetpack Compose seperti onClick dan onChange.
2. Membedakan pendekatan imperatif pada XML/View dan pendekatan deklaratif pada Jetpack Compose ketika tampilan berubah.
3. Memahami perbedaan variabel biasa dan state dalam memicu recomposition pada Compose.
4. Menggunakan remember dan mutableStateOf dalam pengelolaan state.
5. Memahami penggunaan property delegation (by) pada state Compose.
6. Memahami penggunaan Intent implicit pada aplikasi Android berbasis Compose.

Dasar Teori

Event Handling pada Jetpack Compose

Dalam Jetpack Compose, event adalah aksi yang terjadi akibat interaksi pengguna dengan UI, seperti:

menekan tombol memicu onClick pada button

mengetik pada input memicu onChange pada TextField

memilih item

Contoh event yang sering digunakan:

```
Button(onClick = { /* aksi */ }) {  
    Text("Klik")  
}
```

Paradigma Imperatif dan Deklaratif pada Antarmuka

Pada sistem UI berbasis XML/View, perubahan tampilan umumnya dilakukan secara imperatif. Program mencari objek View, kemudian memerintahkan objek tersebut untuk mengubah propertinya. Contohnya, ketika tombol diklik, program memperoleh referensi TextView melalui

```
findViewById(), lalu mengganti nilai text secara langsung.  
val tvMessage = findViewById<TextView>(R.id.tvMessage)  
val btnChange = findViewById<Button>(R.id.btnChange)  
  
btnChange.setOnClickListener {  
    tvMessage.text = "Tombol sudah diklik"  
}
```

```
}
```

Cara berpikir imperatif adalah: "ambil komponen UI tersebut, kemudian ubah propertinya". Pengembang bertanggung jawab menentukan komponen mana yang harus diperbarui dan perintah apa yang harus dijalankan.

Jetpack Compose menggunakan pendekatan deklaratif. Pengembang tidak mengubah objek Text secara langsung. Pengembang mendeklarasikan bahwa teks yang tampil berasal dari suatu state. Ketika state berubah, Compose menjalankan recomposition pada bagian UI yang bergantung pada state tersebut.

```
var message by remember { mutableStateOf("Belum diklik") }

Text(text = message)
Button(onClick = {
    message = "Tombol sudah diklik"
}) {
    Text("Ubah Teks")
}
```

Cara berpikir deklaratif adalah: "UI harus menggambarkan state saat ini". Hubungan tersebut dapat diringkas sebagai $UI = f(state)$. Event seperti `onClick` tidak memerintahkan Text untuk berubah, tetapi mengubah state yang menjadi sumber data bagi UI.

Perbandingan inti:

- Imperatif: event -> cari View -> ubah properti View.
- Deklaratif: event -> ubah state -> Compose memperbarui UI melalui recomposition.

State pada Jetpack Compose

Jetpack Compose menggunakan state-driven UI. Artinya, tampilan merupakan hasil dari state saat ini. Agar Compose mengetahui bahwa data telah berubah dan perlu melakukan recomposition, data tersebut harus menggunakan state yang dapat diamati, misalnya `mutableStateOf()`. Variabel lokal biasa dapat berubah nilainya, tetapi perubahan tersebut tidak otomatis meminta Compose menggambar ulang UI. Contoh:

```
var counter by remember { mutableStateOf(0) }
```

Ketika nilai counter berubah, Compose akan melakukan recomposition.

Remember pada Jetpack Compose

Remember digunakan untuk menyimpan nilai state selama composable masih berada dalam composition. Contoh:

```
val count = remember { mutableStateOf(0) }
```

Jika tidak menggunakan remember, maka nilai akan kembali ke nilai awal setiap recomposition.

Property Delegation (by)

Kotlin menyediakan fitur property delegation. Sederhananya, Property Delegation adalah cara untuk "menitipkan" logika getter dan setter sebuah variabel ke objek lain.

Tanpa by:

```
val count = remember { mutableStateOf(0) }
Text("${count.value}")
count.value++
```

Dengan by:

```
var count by remember { mutableStateOf(0) }

Text("$count")
count++
```

Penggunaan by membuat kode lebih sederhana. Import yang diperlukan :

```
import androidx.compose.runtime.*
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
```

Intent pada Android

Intent digunakan untuk berpindah activity atau memanggil aplikasi lain. Jenis Intent:

- Explicit Intent, digunakan untuk berpindah ke activity tertentu dalam aplikasi. Contoh: `Intent(this, SecondActivity::class.java)`
- Pada Jetpack Compose modern biasanya menggunakan satu Activity saja. Kemudian berpindah layar menggunakan Navigation Compose, sehingga perpindahan halaman tidak lagi mengandalkan intent
- Implicit Intent, digunakan untuk membuka aplikasi lain seperti telepon, browser, whatsapp dan email. Contoh aplikasi telepon: `Intent(Intent.ACTION_DIAL)`

Rangkuman

Event Handling pada Jetpack Compose digunakan untuk menangani interaksi pengguna, seperti `onClick` pada Button dan `onValueChange` pada `OutlinedTextField`.

Pada pendekatan imperatif berbasis XML/View, program memperbarui komponen UI secara langsung, misalnya dengan mengubah properti text pada `TextView`.

Pada pendekatan deklaratif berbasis Compose, program mengubah state dan mendeklarasikan tampilan berdasarkan state tersebut. Compose kemudian memperbarui bagian UI yang terpengaruh.

State adalah data yang dapat memicu perubahan tampilan UI. Jika nilai state berubah, maka Compose akan melakukan recomposition sehingga UI diperbarui.

Perubahan pada variabel biasa tidak akan memperbarui UI, sedangkan perubahan pada state (mutableStateOf) akan memperbarui tampilan.

remember digunakan untuk menyimpan nilai state selama composable masih berada dalam composition.

Penggunaan by pada state membuat kode lebih sederhana karena tidak perlu menggunakan .value.

Intent digunakan untuk menjalankan komponen lain.

- Explicit Intent: membuka activity dalam aplikasi.
- Implicit Intent: membuka aplikasi lain seperti telepon atau WhatsApp.

Alat dan Bahan Praktik

Perangkat keras

- Komputer/Laptop dengan RAM minimal 8 GB, 16 GB direkomendasikan.
- Processor Dual Core.
- Perangkat Android fisik, untuk menjalankan program jika RAM komputer yang digunakan tidak memadai untuk menjalankan Android Emulator di dalam Android Studio.

Perangkat Lunak

- Android Studio dengan Android SDK dan Android Emulator yang telah terinstall.
- Materi Pembelajaran Kelas.
- Modul Praktikum.

Percobaan

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Pra-Percobaan

Pada percobaan ini dibuat dua project terpisah dengan hasil akhir yang sama. Teks awal adalah "Belum diklik". Setelah tombol "Ubah Teks" ditekan, teks berubah menjadi "Tombol sudah diklik". Perbedaan yang diamati bukan hasil akhirnya, melainkan cara program memperbarui UI.

Project A - XML/View (Imperatif)

1. Buat project Android Studio baru menggunakan template Empty Views Activity. Beri nama project ImperativeXmlApp dan pilih Kotlin.
2. Buka res/layout/activity_main.xml, kemudian gunakan kode berikut:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvMessage"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Belum diklik"
        android:textSize="24sp"
        android:layout_marginBottom="16dp" />

    <Button
        android:id="@+id/btnChange"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Ubah Teks" />
</LinearLayout>

```

3. Buka MainActivity.kt. Sesuaikan package dengan package project, kemudian gunakan kode berikut:

```

package com.example.imperativexmlapp

import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val tvMessage = findViewById<TextView>(R.id.tvMessage)
        val btnChange = findViewById<Button>(R.id.btnChange)

        btnChange.setOnClickListener {
            tvMessage.text = "Tombol sudah diklik"
        }
    }
}

```

4. Jalankan aplikasi. Pastikan teks berubah setelah tombol diklik.

Project B - Jetpack Compose (Deklaratif)

1. Buat project Android Studio baru menggunakan template Empty Activity untuk Jetpack Compose. Beri nama project DeclarativeComposeApp dan pilih Kotlin.

2. Buka MainActivity.kt, pertahankan package project, kemudian gunakan fungsi composable berikut:

```
@Composable
fun ChangeTextScreen() {
    var message by remember {
        mutableStateOf("Belum diklik")
    }

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = message, fontSize = 24.sp)
        Button(onClick = {
            message = "Tombol sudah diklik"
        }) {
            Text("Ubah Teks")
        }
    }
}
```

3. Tambahkan import yang diperlukan:

```
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.sp
```

4. Pada fungsi onCreate(), ubah isi setContent agar memanggil ChangeTextScreen():

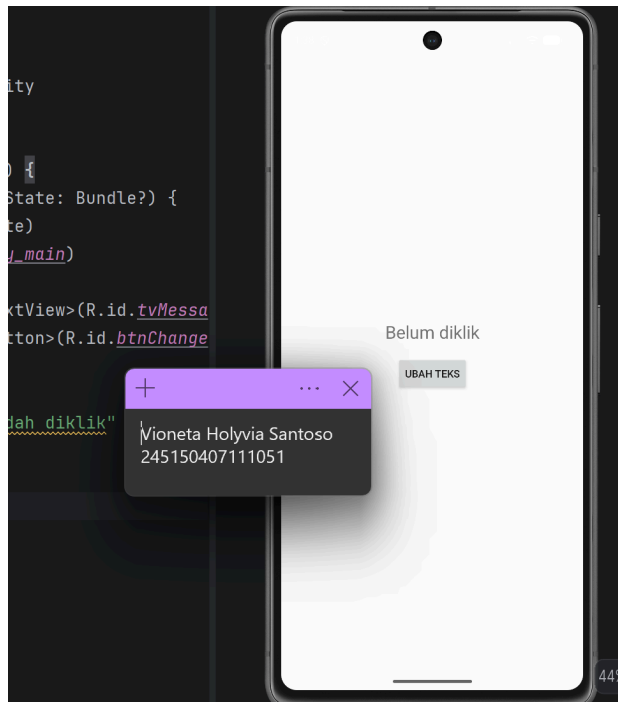
```
setContent {
    ChangeTextScreen()
}
```

5. Jalankan aplikasi. Pastikan hasil tampilannya sama dengan project XML/View.

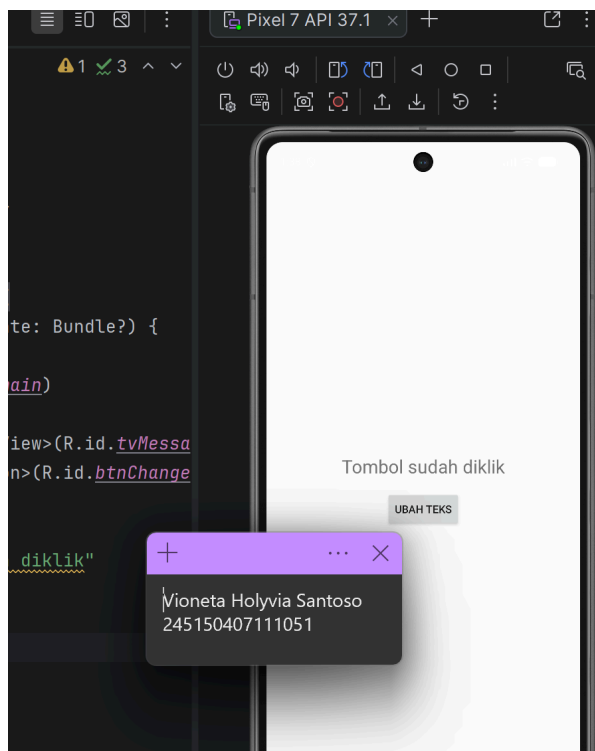
Aktivitas Percobaan

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Sebelum diklik (XML) :



Sesudah diklik:



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang diubah secara langsung adalah komponen UI **TextView (tvMessage)**.

```
btnChange.setOnClickListener {
    tvMessage.text = "Tombol sudah diklik"
```

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Nilai yang diubah adalah nilai dari variabel state `message`

```
Button(onClick = {
    message = "Tombol sudah diklik"
}) {
```

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Alasan tidak terdapat perintah seperti `tvMessage.text` atau `setText()` Dalam Jetpack Compose karena tampilan dikelola menggunakan pendekatan deklaratif di mana UI merupakan fungsi dari state ($\$UI = f(state)\$$). Kita tidak memegang referensi langsung ke komponen UI untuk mengubah nilainya secara manual, melainkan hanya perlu mengubah nilai state. Compose secara otomatis akan merender ulang (recomposition) tampilan yang terikat pada state tersebut

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Alur perubahan pada XML/View (Imperatif)

Ketika **event `onClick`** dipencet : program akan mencari komponen UI di layar menggunakan **`findViewById()`** (misalnya `tvMessage`), kemudian program secara langsung mengubah properti teks pada komponen tersebut (`tvMessage.text = ...`) sehingga **tampilan di layar berubah**.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Alur perubahan pada Compose (Deklaratif)

Ketika **event `onClick`** dipencet : program tidak mengubah komponen UI secara langsung, melainkan mengubah **nilai dari state** (`message = ...`). dimana perubahan ini akan memicu proses ***recomposition*** yang membuat Compose secara otomatis **menggambar ulang** bagian UI yang terikat dengan *state* tersebut sehingga **tampilannya diperbarui**.

7. Apa yang terjadi jika `message` diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Nilai variabel `message` di dalam memori akan berubah saat tombol diklik, **tetapi tampilan UI tidak akan berubah**. Hal ini terjadi karena variabel biasa tidak dapat diamati oleh Compose (*observable state*), sehingga Compose tidak menerima notifikasi untuk memicu proses *recomposition*

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Menurut saya, Imperatif lebih berfokus pada *bagaimana* cara mengubah UI (mengambil referensi komponen tampilan dan memperbarui propertinya secara manual satu per satu). Sedangkan Deklaratif, berfokus pada *apa* yang harus ditampilkan berdasarkan state saat ini (mendefinisikan UI berbasis data, lalu memperbarui data/state agar sistem memperbarui UI secara otomatis).

Event Handling State

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity:

```
@Composable
fun EventStateTest() {
    var count = 0
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```

3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.Button
```

```
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
```

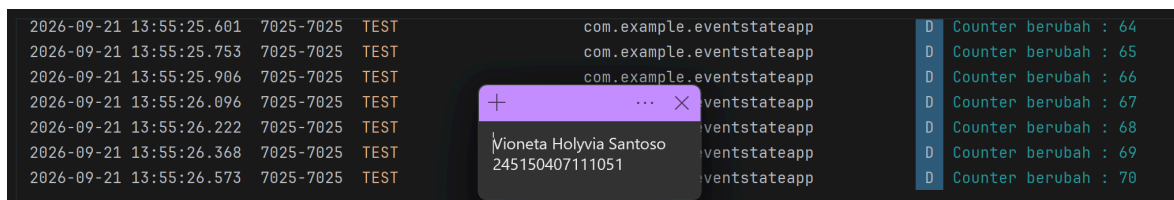
4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

```
setContent {
    EventStateTest()
}
```

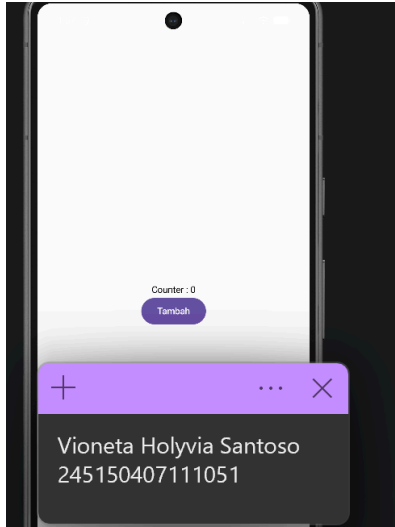
5. Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-compile.

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
muncul pesan log dengan nilai angka *counter* yang terus bertambah setiap kali tombol diklik pada LogCat
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
Tampilan tidak ada yang berubah. Counter masih menunjukkan angka 0 meskipun tombol tambah sudah dipencet berkali kali



4. Apa yang menyebabkan hal tersebut terjadi?

Hal ini terjadi karena variabel `count` dibuat menggunakan variabel lokal biasa (`var count = 0`), bukan `State` yang dapat diamati oleh Jetpack Compose (`mutableStateOf`). Karena bukan sebuah state, Compose tidak menerima notifikasi ketika nilai `count` bertambah di memori, sehingga Compose tidak memicu proses *recomposition* untuk menggambar ulang tampilan di layar.

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable `EventStateTest`.
6. Ubah kode program pada bagian fungsi Composable `EventStateTest` dari yang sebelumnya:

`var count = 0`

menjadi:

`var count by remember { mutableStateOf(0) }`

7. Berikut kode lengkap `EventStateTest`:

```
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
        }) {
            "+"
        }
    }
}
```

```

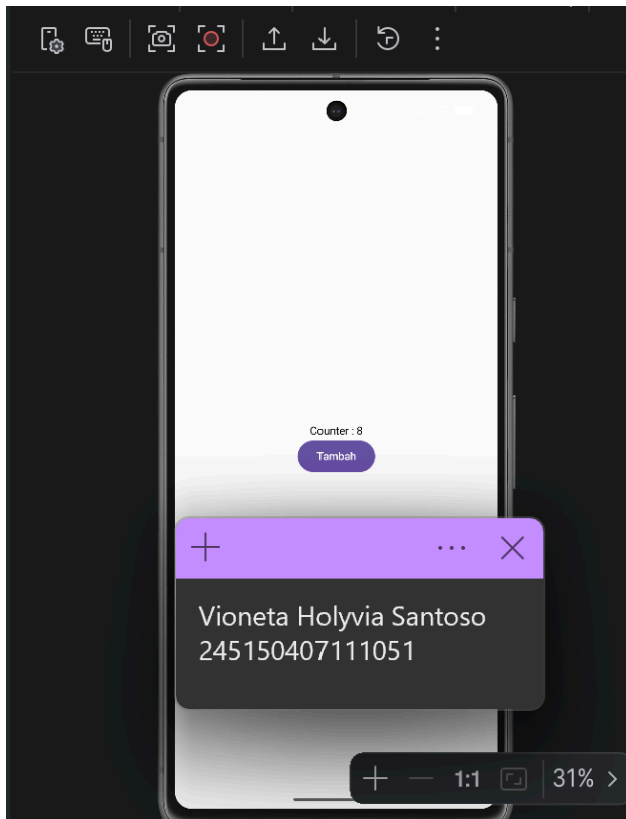
        Log.d("TEST", "Counter berubah : $count")
    }) {
        Text("Tambah")
    }
}
}

```

8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
Sama seperti sebelumnya, muncul pesan log dengan nilai angka *counter* yang terus bertambah setiap kali tombol diklik pada LogCat
10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
Teks pada layar aplikasi berubah secara *real-time* mengikuti jumlah klik tombol



11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Perubahan ini terjadi karena penggunaan `mutableStateOf(0)` membuat variabel `count` menjadi sebuah **Observable State** yang nilainya terus dipantau oleh Jetpack Compose. Kata kunci `remember` berfungsi untuk mempertahankan/menyimpan nilai state tersebut di dalam memori selama proses *recomposition*. Ketika nilai `count` bertambah, Compose mendeteksi perubahan state dan secara otomatis memperbarui tampilan UI di layar

12. Apa yang dimaksud recomposition pada Compose?

Recomposition adalah proses dipanggilnya kembali fungsi-fungsi Composable oleh Jetpack Compose ketika terjadi perubahan pada *state* yang terikat dengan tampilan tersebut, sehingga tampilan UI dapat diperbarui dengan nilai data yang baru.

13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable `EventStateTest`. Kemudian di dalam `Column` panggil composable `OutlinedTextField` di bawah `Button` Tambah. Penambahan kode program membuat kode pada fungsi Composable `EventStateTest` menjadi:

```
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    var text by remember { mutableStateOf("") }

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
        OutlinedTextField(
            value = text,
            onChange = {
                text = it
                Log.d("TEST", "Text berubah : $it")
            },
            label = { Text("Text Field") }
        )
    }
}
```

```
}
}
```

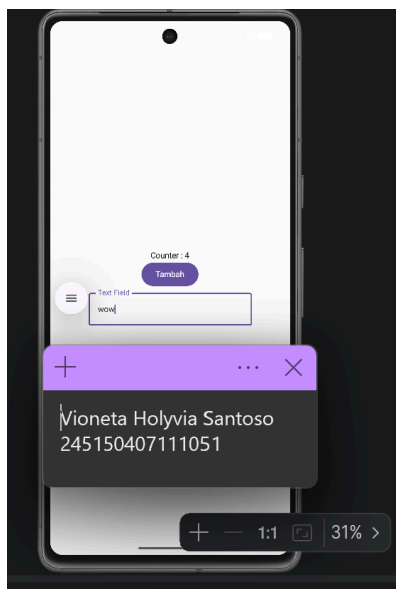
14. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

LogCat langsung mencetak log berisi karakter atau teks terbaru yang diketikkan, contohnya:

```
2026-09-21 14:11:58.581 8109-8109 TEST com.example.eventstateapp D Text berubah : apa
2026-09-21 14:11:59.658 8109-8109 TEST com.example.eventstateapp D Text berubah : ap
2026-09-21 14:12:00.193 8109-8109 TEST com.example.eventstateapp D Text berubah : a
2026-09-21 14:12:00.238 8109-8109 TEST com.example.eventstateapp D Text berubah :
2026-09-21 14:12:00.501 8109-8109 TEST com.example.eventstateapp D Text berubah : w
2026-09-21 14:12:00.576 8109-8109 TEST com.example.eventstateapp D Text berubah : wo
2026-09-21 14:12:00.717 8109-8109 TEST com.example.eventstateapp D Text berubah : wow
```

15. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Huruf atau karakter yang diketikkan pengguna langsung muncul di dalam kotak `OutlinedTextField` secara *real-time*



16. Apa yang menyebabkan hal tersebut terjadi?

Ketika ada input dari keyboard, event `onValueChange` dipanggil dan memperbarui nilai state `text` (`text = it`). Perubahan nilai state `text` ini memicu *recomposition* pada `OutlinedTextField` sehingga tampilan parameter `value = text` diperbarui dengan teks yang baru.

17. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

- State `count` (tipe `Int`) — menyimpan nilai counter angka
- State `text` (tipe `String`) — menyimpan isi teks inputan dari `OutlinedTextField`.

18. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

- Event `onClick` pada komponen `Button`.
- Event `onValueChange` pada komponen `OutlinedTextField`

19. Kapan event - event tersebut dijalankan?

- `onClick`: Dijalankan saat pengguna menekan/mengetuk tombol "Tambah".
- `onValueChange`: Dijalankan setiap kali pengguna mengetik, menambah, atau menghapus karakter di dalam kolom `OutlinedTextField`

Implisit Intent

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template Empty Activity untuk Phone and Tablet. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file MainActivity.kt dan tambahkan fungsi composable berikut ini di luar class MainActivity

`@Composable`

```
fun ImplicitIntentTest(context: Context) {  
    Column(  
        verticalArrangement = Arrangement.Center,  
        horizontalAlignment = Alignment.CenterHorizontally,  
        modifier = Modifier.fillMaxSize()  
    ) {  
        // Buka Browser  
        Button(onClick = {  
            val intent = Intent(Intent.ACTION_VIEW).apply {  
                data = Uri.parse("https://www.google.com")  
            }  
            context.startActivity(intent)  
        }) {  
            Text("Buka Browser")  
        }  
        Spacer(modifier = Modifier.height(16.dp))  
        // Buka Dial Telepon  
        Button(onClick = {  
            val intent = Intent(Intent.ACTION_DIAL).apply {  
                data = Uri.parse("tel:08123456789")  
            }  
            context.startActivity(intent)  
        }) {  
            Text("Buka Telepon")  
        }  
    }  
}
```

```

        Spacer(modifier = Modifier.height(16.dp))
        // Kirim Email (dengan chooser)
        Button(onClick = {
            val intent = Intent(Intent.ACTION_SENDTO).apply {
                data = Uri.parse("mailto:test@gmail.com")
            }
            val chooser = Intent.createChooser(intent, "Pilih aplikasi
email")
            context.startActivity(chooser)
        }) {
            Text("Kirim Email")
        }
        Spacer(modifier = Modifier.height(16.dp))
        // Buka Maps
        Button(onClick = {
            val intent = Intent(Intent.ACTION_VIEW).apply {
                data = Uri.parse("geo:-7.250445,112.768845")
            }
            context.startActivity(intent)
        }) {
            Text("Buka Maps")
        }
    }
}

```

3. Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```

import android.content.Context
import android.content.Intent
import android.net.Uri
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

```

4. Buka file MainActivity.kt, temukan fungsi onCreate() yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah isi setContent{ } menjadi

```

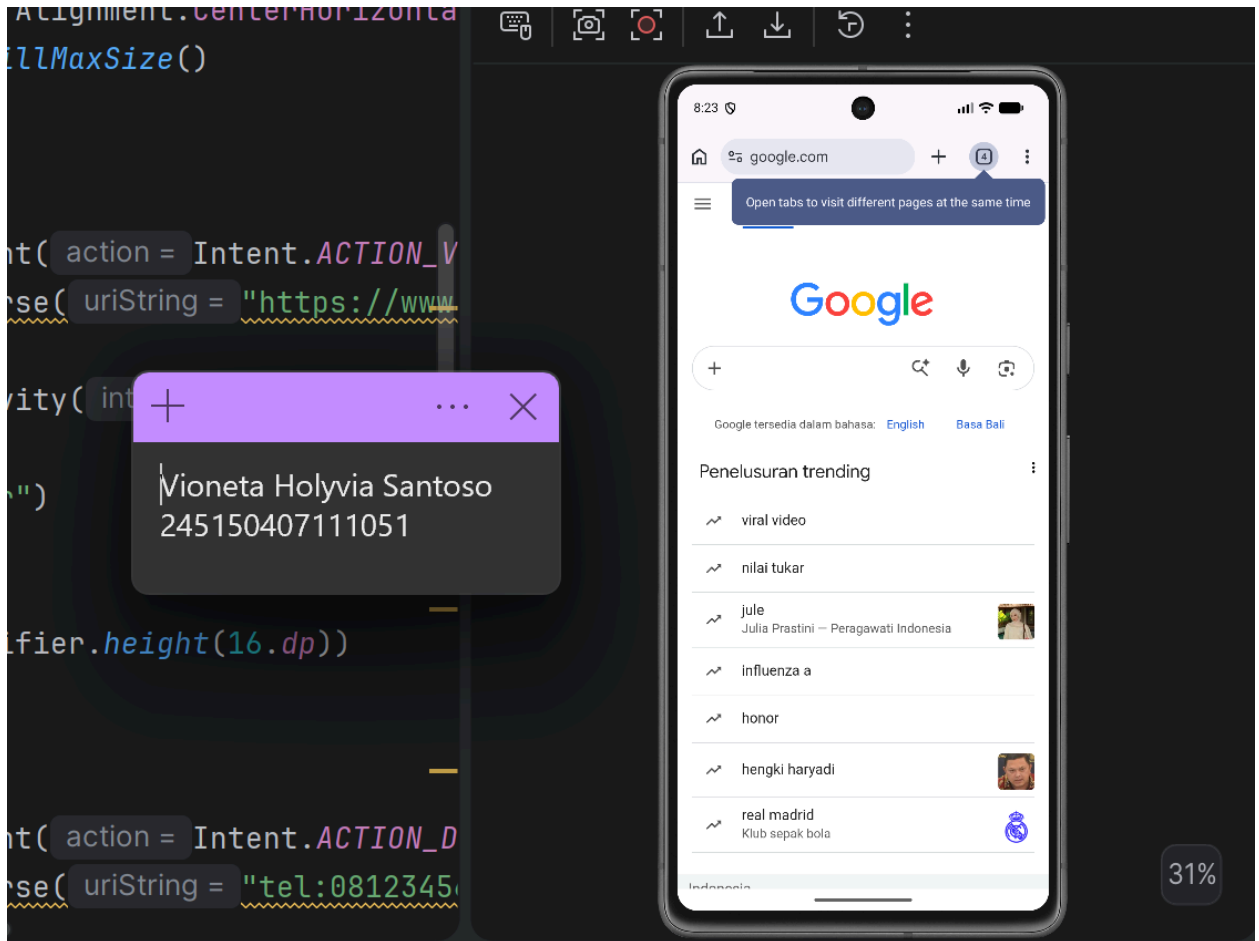
setContent {
    val context = LocalContext.current

```

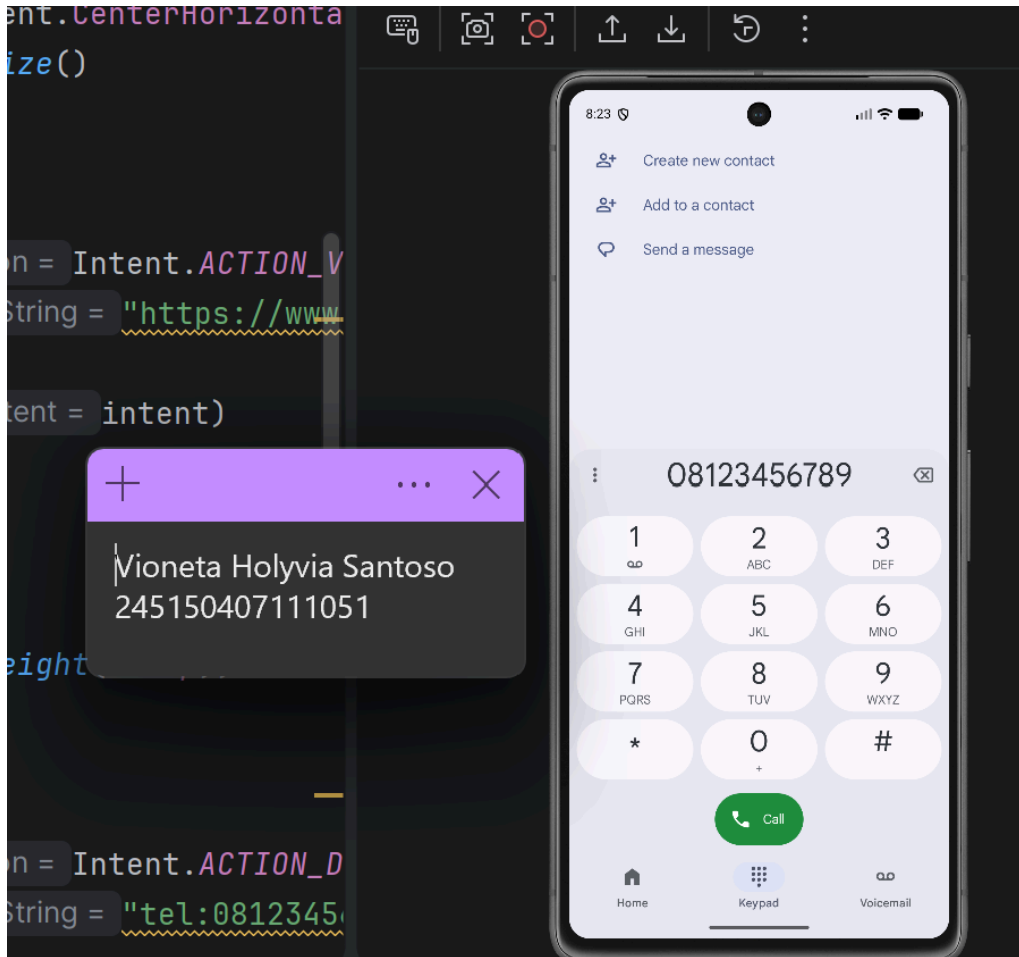
```
ImplicitIntentTest(context)
}
```

Aktivitas Percobaan

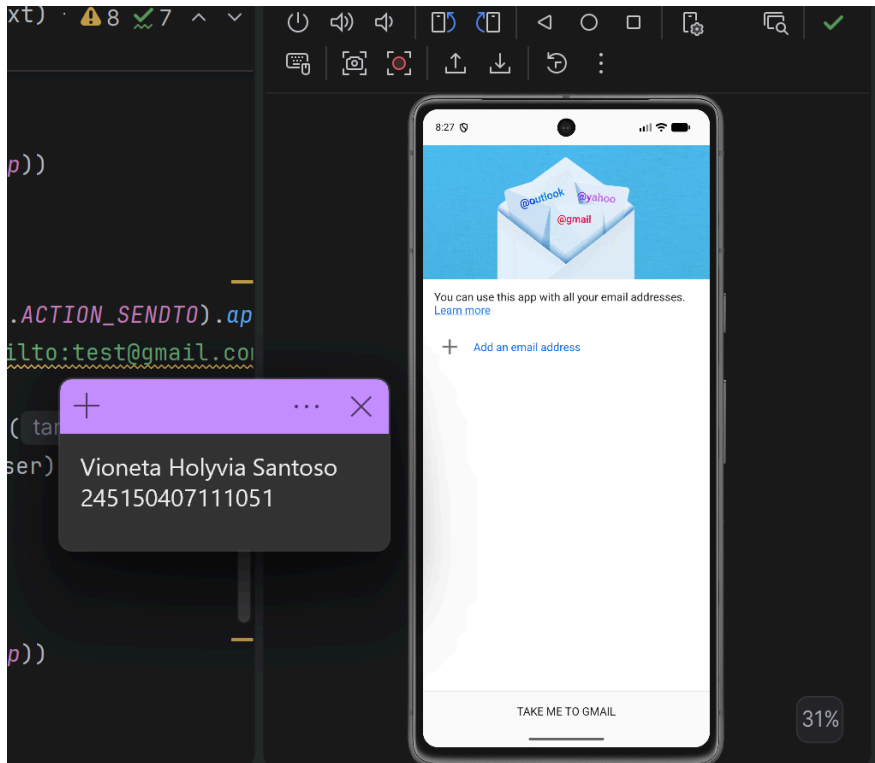
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya
2. Klik Buka Browser



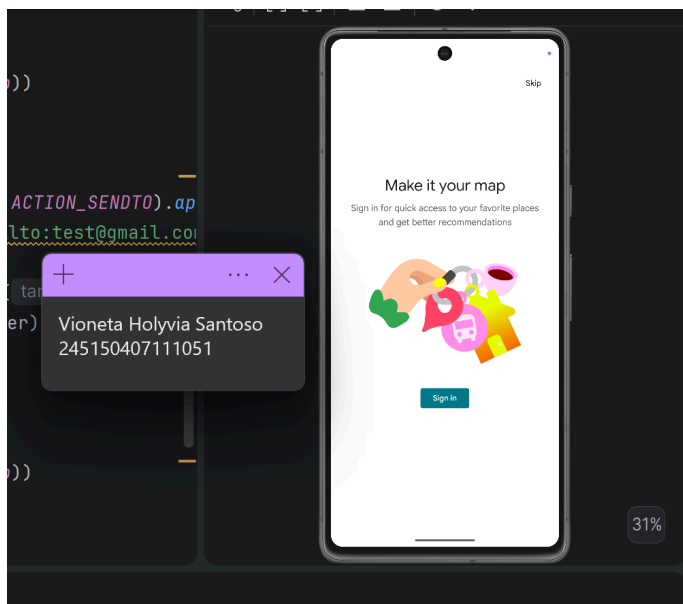
3. Klik Buka Telepon



4. Klik Kirim Email



5. Klik Buka Maps



Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

6. Pada kode yang digunakan, terdapat beberapa action seperti:

Intent.ACTION_VIEW

Intent.ACTION_DIAL

Intent.ACTION_SENDTO

7. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Intent.ACTION_VIEW: Digunakan untuk menampilkan data kepada pengguna. Jenis tampilan menyesuaikan data URI yang diberikan, misalnya menampilkan situs web atau titik lokasi pada peta.

Intent.ACTION_DIAL: Digunakan untuk membuka aplikasi papan ketik telepon (dialer) dan langsung mengisi nomor tujuan tanpa memulai panggilan secara otomatis.

Intent.ACTION_SENDTO: Digunakan untuk mengirimkan pesan/data ke tujuan spesifik yang ditentukan oleh skema URI, umumnya digunakan khusus untuk mengarahkan pengguna ke aplikasi email.

8. Pada kode yang digunakan, terdapat beberapa data seperti:

```
data = Uri.parse("mailto:test@gmail.com")
```

```
data = Uri.parse("https://www.google.com")
```

```
data = Uri.parse("geo:-7.250445,112.768845")
```

Apa fungsi data pada intent ?

Data pada Intent berfungsi memberikan muatan informasi atau alamat tujuan spesifik menggunakan format URI (Uniform Resource Identifier). Skema awalan URI memberitahu sistem jenis data yang akan diolah:

- **https://** mengindikasikan halaman web.
- **tel:** mengindikasikan nomor telepon.
- **mailto:** mengindikasikan alamat email.
- **geo:** mengindikasikan koordinat peta/geografis.

9. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Sistem Android menggunakan pasangan Action dan skema Data untuk mencocokkan *Intent Filter* dari aplikasi-aplikasi yang terpasang di perangkat:

- Jika Action-nya **ACTION_VIEW** dengan Data **https://**, Android menyimpulkan bahwa aksi tersebut butuh Browser.
- Jika Action-nya **ACTION_DIAL** dengan Data **tel:**, Android menyimpulkan aplikasi penanganan yang cocok adalah Aplikasi Telepon.
- Jika Action-nya **ACTION_SENDTO** dengan Data **mailto:**, Android memunculkan Aplikasi Email.
- Jika Action-nya **ACTION_VIEW** dengan Data **geo:**, Android mengarahkan pengguna ke Aplikasi Peta/Maps.

Link Github : <https://github.com/neyney281/pemesanantikett.git>

Referensi

Google (2024) The Activity Lifecycle. Available at:

<https://developer.android.com/guide/components/activities/activity-lifecycle>

Google (2024) Jetpack Compose layouts. Available at:

<https://developer.android.com/jetpack/compose/layouts>

Google (2024) Compose Modifiers. Available at:

<https://developer.android.com/jetpack/compose/modifiers>

Google Developers (2026) Thinking in Compose. Available at:

<https://developer.android.com/develop/ui/compose/mental-model> (Accessed: 8 September 2026).

Google Developers (2026) State and Jetpack Compose. Available at:

<https://developer.android.com/develop/ui/compose/state> (Accessed: 8 September 2026).

Glosarium

Event

Aksi atau kejadian yang dipicu oleh interaksi pengguna atau sistem, seperti klik tombol, input teks, atau perubahan data, yang kemudian akan memicu perubahan pada aplikasi.

Delegation (Property Delegation)

Mekanisme di Kotlin untuk mendelegasikan pengelolaan suatu properti ke objek lain, biasanya menggunakan kata kunci `by`.

Intent

Objek pada Android yang digunakan untuk melakukan komunikasi antar komponen aplikasi atau dengan aplikasi lain untuk menjalankan suatu aksi tertentu.

Implicit Intent

Intent yang tidak menyebutkan secara langsung komponen tujuan, melainkan hanya mendefinisikan aksi dan data sehingga sistem Android akan memilih aplikasi yang sesuai.

Imperatif

Pendekatan pemrograman UI yang memperbarui komponen tampilan secara langsung melalui referensi objek dan perubahan propertinya.

Deklaratif

Pendekatan pemrograman UI yang mendeskripsikan tampilan berdasarkan state saat ini; ketika state berubah, framework memperbarui UI yang diperlukan.

State

Data atau nilai yang merepresentasikan kondisi saat ini dari UI.

Dalam Jetpack Compose, perubahan state akan secara otomatis menyebabkan UI diperbarui (recomposition).